

PRUEBAS DE TESTING DE ODONTOGATE

ESTUDIANTE: Ashlee Tatiana Martínez Garzón

ENTIDADES: Appointment y Schedule

ruta de acceso:

C:\Proyecto_OdontoGate\OdontoGate\src\test\java\com\OdontoGate\ArtefactoOdontoGate\service\AppointmentServiceTest.java

TEST DE FUNCIONALIDADES:

1. No se puede agendar una cita en un horario no disponible:
 - a. Código:

```
@Test
void cuandoHorarioNoDisponible_debeLanzarExcepcion() {

    AppointmentRequest request = new AppointmentRequest();
    request.setPatientId(1);
    request.setDoctorId(1);
    request.setDate(LocalDate.now().plusDays(1));
    request.setTime(LocalTime.of(9, 0));
    request.setDoctorScheduleId(1);
    request.setStatus("pendiente");
    request.setReason("Revisión");
    request.setModifiedBy(1);

    Patient patient = new Patient();
    patient.setId(1);

    Doctor doctor = new Doctor();
    doctor.setId(1);

    Schedule schedule = new Schedule();
    schedule.setId(1);
    schedule.setIsAvailable(false);

    when(patientRepository.findById(1)).thenReturn(Optional.of(patient));

    when(doctorRepository.findById(1)).thenReturn(Optional.of(doctor));

    when(scheduleRepository.findById(1)).thenReturn(Optional.of(schedule));

    RuntimeException ex = assertThrows(RuntimeException.class,
        () -> appointmentService.create(request));

    assertEquals("El horario seleccionado no está disponible",
        ex.getMessage());
}
```

b. Caso límite: El horario existe pero está marcado como no disponible

2. No se pueden agendar dos citas al mismo tiempo y con el mismo doctor:

a. Código:

```
@Test
void cuandoYaExisteCitaMismoHorario_debeLanzarExcepcion() {
    AppointmentRequest request = new AppointmentRequest();
    request.setPatientId(1);
    request.setDoctorId(1);
    request.setDate(LocalDate.now().plusDays(1));
    request.setTime(LocalTime.of(9, 0));
    request.setDoctorScheduleId(1);
    request.setStatus("pendiente");
    request.setReason("Revisión");
    request.setModifiedBy(1);

    Patient patient = new Patient();
    patient.setId(1);

    Doctor doctor = new Doctor();
    doctor.setId(1);

    Schedule schedule = new Schedule();
    schedule.setId(1);
    schedule.setIsAvailable(true);

    // Cita existente con el mismo doctor, fecha y hora
    Appointment citaExistente = new Appointment();
    citaExistente.setId(99);

    when(patientRepository.findById(1)).thenReturn(Optional.of(patient));

    when(doctorRepository.findById(1)).thenReturn(Optional.of(doctor));

    when(scheduleRepository.findById(1)).thenReturn(Optional.of(schedule));

    when(appointmentRepository.findByDoctorIdAndDateAndTime(1,
        request.getDate(), request.getTime()))
        .thenReturn(List.of(citaExistente));

    RuntimeException ex = assertThrows(RuntimeException.class,
        () -> appointmentService.create(request));
}
```

```

    assertEquals("El doctor ya tiene una cita agendada en esa fecha y hora", ex.getMessage());
}

```

- b. Caso límite: El horario está disponible, pero el doctor ya tiene una cita registrada en esa misma fecha y hora.

3. No se puede agendar una cita con fecha pasada:

- a. Código:

```

@Test
void cuandoFechaEsPasada_debeLanzarExcepcion() {
    AppointmentRequest request = new AppointmentRequest();
    request.setPatientId(1);
    request.setDoctorId(1);
    request.setDate(LocalDate.now().minusDays(1)); // fecha
pasada
    request.setTime(LocalTime.of(9, 0));
    request.setDoctorScheduleId(1);
    request.setStatus("pendiente");
    request.setReason("Revisión");
    request.setModifiedBy(1);

    Patient patient = new Patient();
    patient.setId(1);

    Doctor doctor = new Doctor();
    doctor.setId(1);

    when(patientRepository.findById(1)).thenReturn(Optional.of(patient));
    when(doctorRepository.findById(1)).thenReturn(Optional.of(doctor));

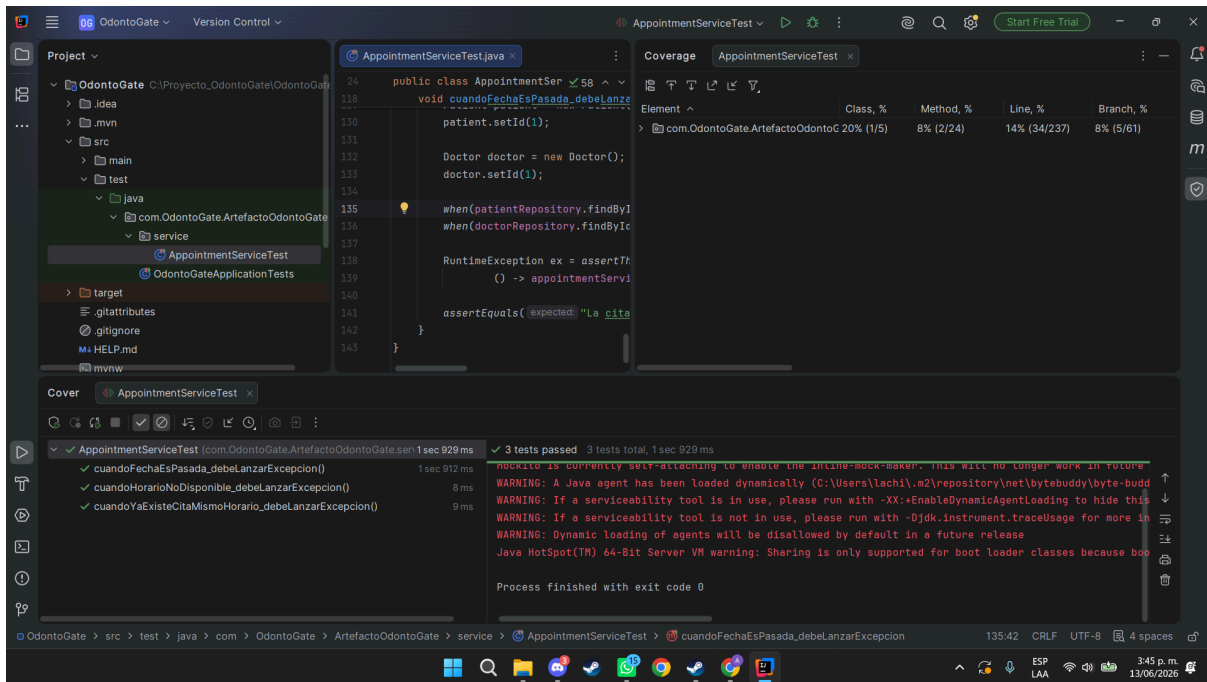
    RuntimeException ex = assertThrows(RuntimeException.class,
        () -> appointmentService.create(request));

    assertEquals("La cita debe agendarse en una fecha futura",
ex.getMessage());
}
}

```

- b. Caso límite: La validación se detiene cuando la fecha es un día anterior más que el actual.

RUN APPOINTMENTSERVICETEST CON COVERAGE:



ESTUDIANTE: Carolina Castro Galarza

ENTIDADES: Payment

ruta de acceso:

C:\Proyecto_OdontoGate\OdontoGate\src\test\java\com\OdontoGate\ArtefactoOdontoGate\service\PaymentServiceTest.java

TEST DE FUNCIONALIDADES:

4. No se puede pagar una cita que ya se pagó :
 - a. Código:

```
@Test
void cuandoCitaYaTienePago_debeLanzarExcepcion() {

    PaymentRequest request = new PaymentRequest();
    request.setAppointmentId(1);
    request.setAmount(new BigDecimal("50000"));
    request.setMethod("EFECTIVO");

    Appointment appointment = new Appointment();
    appointment.setId(1);

    Payment pagoExistente = new Payment();
    pagoExistente.setId(1);
    pagoExistente.setStatus("PENDIENTE");

    when(appointmentRepository.findById(1))
        .thenReturn(Optional.of(appointment));
    when(paymentRepository.findById(1))
```

```

        .thenReturn(Optional.of(pagoExistente));

RuntimeException ex = assertThrows(RuntimeException.class,
    () -> paymentService.createPayment(request));

assertEquals("Esta cita ya tiene un pago", ex.getMessage());
}

```

b. Casos borde:

- El estado del pago es PAGADO

5. No se puede pagar una cita que no existe :

a. Código:

```

@Test
void cuandoCitaNoExiste_debeLanzarExcepcion() {

    PaymentRequest request = new PaymentRequest();
    request.setAppointmentId(99);
    request.setAmount(new BigDecimal("50000"));
    request.setMethod("EFECTIVO");

    when(appointmentRepository.findById(99))
        .thenReturn(Optional.empty());

    RuntimeException ex = assertThrows(RuntimeException.class,
        () -> paymentService.createPayment(request));

    assertEquals("Cita no encontrada", ex.getMessage());
}

```

b. Casos borde:

- El id de la cita no es encontrado, es decir, no existe.

6. Un monto no puede ser negativo :

a. Código:

```

@Test

```

```

void cuandoMontoEsNegativo_debeLanzarExcepcion() {

    // 1. Preparar datos
    PaymentRequest request = new PaymentRequest();
    request.setAppointmentId(1);
    request.setAmount(new BigDecimal("-50000"));
    request.setMethod("EFECTIVO");

    // 2. Ejecutar y verificar
    RuntimeException ex = assertThrows(RuntimeException.class,
        () -> paymentService.createPayment(request));

    assertEquals("El monto no puede ser negativo", ex.getMessage());
}

```

b. Casos borde:

- El monto puesto es un valor negativo.

FUNCIONAMIENTO DE LOS 3 TESTS + COVERAGE

The screenshot shows an IDE with the following components:

- Project Explorer:** Shows the project structure with folders like 'resources', 'test', and 'target'.
- Code Editor:** Displays the source code for `PaymentServiceTest.java`, including the `cuandoMontoEsNegativo_debeLanzarExcepcion()` test method.
- Coverage View:** A table showing coverage for various classes and methods.

Element	Class, %	Method, %	Line, %	Branch, %
com.OdontoGate.ArtefactoOdontoGate	12% (1/8)	2% (1/42)	1% (6/339)	4% (3/69)
AppointmentService	0% (0/1)	0% (0/9)	0% (0/94)	0% (0/22)
LoginService	0% (0/1)	0% (0/3)	0% (0/36)	0% (0/12)
MedicalRecordService	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
MedicalRecordServiceImpl	0% (0/1)	0% (0/7)	0% (0/35)	0% (0/2)
PaymentService	100% (1/1)	14% (1/7)	14% (6/41)	75% (3/4)
ReceiptService	0% (0/1)	0% (0/4)	0% (0/26)	0% (0/2)
ScheduleService	0% (0/1)	0% (0/8)	0% (0/48)	0% (0/14)
UserService	0% (0/2)	0% (0/4)	0% (0/59)	0% (0/13)
- Test Results:** Shows that 3 tests passed out of 3 tests total, with a total execution time of 1 second and 728 milliseconds.
- Console:** Displays warnings from Mockito and Java HotSpot VM, and a message indicating the process finished with exit code 0.

ESTUDIANTE: Boris Mateo Paez Jimenez

ENTIDADES: User, Doctor, Patient, Administrator

ruta de acceso:

C:\Proyecto_OdontoGate\OdontoGate\src\test\java\com\OdontoGate\ArtefactoOdontoGate\service\UserServiceCreateTest.java

C:\Proyecto_OdontoGate\OdontoGate\src\test\java\com\OdontoGate\ArtefactoOdontoGate\service\UserServiceDeleteTest.java

TEST DE FUNCIONALIDADES:

1. No se puede crear un usuario cuando el correo ya se encuentra registrado.

Código:

```
@Test
    void createUser_DeberiaLanzarBadRequest_CuandoEmailYaExiste() {
        CrearUsuarioRequest request = new CrearUsuarioRequest();
        request.setEmail("doctor@test.com");
        request.setUserType(UserType.DOCTOR);

        when(userRepository.existsByEmail("doctor@test.com")).thenReturn(true);

        ResponseStatusException exception = assertThrows(
            ResponseStatusException.class,
            () -> userService.createUser(request)
        );

        assertEquals(HttpStatus.BAD_REQUEST,
            exception.getStatusCode());
        assertEquals("El email ya está registrado",
            exception.getReason());

        verify(userRepository, never()).save(any(User.class));
    }
}
```

Caso límite: El Email ingresado ya existe en el sistema, por lo que no se guarda ningún usuario.

2. Se puede crear correctamente un usuario tipo Doctor cuando los datos son válidos

Código:

```
@Test
    void createUser_DeberiaCrearDoctor_CuandoDatosSonValidos() {
        CrearUsuarioRequest request = new CrearUsuarioRequest();
        request.setName("Carlos");
        request.setLastname("Perez");
        request.setEmail("doctor@test.com");
        request.setPassword("123456");
        request.setPhone("3001234567");
        request.setUserType(UserType.DOCTOR);
    }
```

```

        request.setSpecialty("Ortodoncia");
        request.setMedicalLicense("MED123");
        request.setPhotoUrl("photo.jpg");

when(userRepository.existsByEmail("doctor@test.com")).thenReturn(false)
;

when(userRepository.save(any(User.class))).thenAnswer(invocation -> {
    User user = invocation.getArgument(0);
    user.setId(1);
    return user;
});

    UsuarioCreadoResponse response =
userService.createUser(request);

    assertNotNull(response);
    assertEquals(1, response.getId());
    assertEquals("Carlos", response.getName());
    assertEquals("Perez", response.getLastname());
    assertEquals("doctor@test.com", response.getEmail());
    assertEquals("3001234567", response.getPhone());
    assertTrue(response.getActive());
    assertEquals(UserType.DOCTOR, response.getUserType());

    verify(userRepository, times(1)).save(any(User.class));
}

```

Caso límite: El email no está registrado y los datos del doctor son válidos, por lo que el sistema guarda el usuario y retorna con los mismos datos ingresados.

3. Se elimina correctamente un usuario tipo Doctor cuando el correo existe en el sistema.

Código:

```

@Test
void deleteUser_DeberiaEliminarDoctor_CuandoCorreoExiste() {
    DeleteUserRequest request = new DeleteUserRequest();
    request.setEmail("doctor@test.com");

    User user = new User();
    user.setId(1);
    user.setEmail("doctor@test.com");
}

```



```

when(userRepository.findByEmail("doctor@test.com")).thenReturn(user);

when(administratorRepository.existsById(1)).thenReturn(false);
when(patientRepository.existsById(1)).thenReturn(false);
when(doctorRepository.existsById(1)).thenReturn(true);

DeleteUserResponse response = userService.deleteUser(request);

assertNotNull(response);
assertEquals("Usuario eliminado correctamente.",
response.getMensaje());
assertEquals("doctor@test.com", response.getEmail());

verify(doctorRepository, times(1)).deleteById(1);
verify(patientRepository, never()).deleteById(1);
verify(administratorRepository, never()).deleteById(1);
verify(userRepository, times(1)).deleteById(1);
}

```

Caso límite: El email existe y el usuario es doctor, por lo que se elimina el registro en la tabla doctor y luego el usuario base.

FUNCIONAMIENTO DE LOS 3 TESTS + COVERAGE (desde VS Code):

The screenshot displays the VS Code interface with the following components:

- TESTING Explorer:** Shows a list of tests and their execution times.

Test Name	Time
ArtefactoOdontoGate	2.9s
com.OdontoGate.ArtefactoOdontoG...	784ms
com.OdontoGate.ArtefactoOdontoG...	29ms
LoginServiceTest	1.3s
PaymentServiceTest	724ms
UserServiceCreateTest	34ms
- TEST COVERAGE:** Shows coverage percentages for various classes.

Class	Coverage
ArtefactoOdontoGate	22.64%
exception	10.00%
PaymentServiceTest	0.00%
ReceiptExceptions.java	0.00%
ScheduleExceptions.java	0.00%
model	100.00%
UserType.java	100.00%
OdontoGateApplication.java	40.00%
service	24.58%
AppointmentService.java	0.00%
LoginService.java	66.67%
MedicalRecordService...	8.33%
PaymentService.java	21.05%
ReceiptService.java	0.00%
ScheduleService.java	0.00%
UserService.java	88.61%
- TEST RESULTS:** Shows the output of the tests, including the test name and the expected results.


```

%TESTE 17,login_DeberiaRetornarPaciente_CuandoCredencialesSonCorrectas(com.OdontoGate.ArtefactoOdontoGate.service.LoginServiceTest)
%TESTS 18,login_DeberiaRetornarDoctor_CuandoCredencialesSonCorrectas(com.OdontoGate.ArtefactoOdontoGate.service.LoginServiceTest)
%TESTE 19,login_DeberiaLanzarBadRequest_CuandoCredencialesSonInvalidas(com.OdontoGate.ArtefactoOdontoGate.service.LoginServiceTest)
%TESTS 19,login_DeberiaLanzarBadRequest_CuandoCredencialesSonInvalidas(com.OdontoGate.ArtefactoOdontoGate.service.LoginServiceTest)
%RUNTIME22522
      
```
- Test Runner for Java:** Shows a list of tests and their execution times.

Test Name	Time
OdontoGateApplicationTests	2.9s
PaymentServiceTest	784ms
UserServiceCreateTest	29ms
LoginServiceTest	1.3s
contextLoads0	724ms
quandoCitaVaTienePago_debeLanzarExcep...	34ms
quandoMontoEsNegativo_debeLanzarExce...	2.9s
createUser_DeberiaCrearPaciente_CuandoDa...	784ms
createUser_DeberiaLanzarBadRequest_Cua...	29ms
deleteUser_DeberiaEliminarDoctor_Cuando...	1.3s

ESTUDIANTE: Kevin Alexander Rodriguez Forero

ENTIDADES: MedicalRecord

RUTA DE ACCESO:

C:\Proyecto_OdontoGate\OdontoGate\src\test\java\com\OdontoGate\ArtefactoOdontoGate\service\MedicalRecordServiceTest.java

TEST DE FUNCIONALIDADES:

1. Verifica que create() mappee correctamente todos los campos del request al response.

Codigo

```
@Test
    @DisplayName("Crea historia clínica y mapea correctamente todos los campos")
    void cuandoDatosSonValidos_debeMapearYRetornarResponse() {
        when(medicalRecordRepository.save(any(MedicalRecord.class)))
            .thenReturn(invocation -> {
                MedicalRecord saved = invocation.getArgument(0);
                saved.setId(EXISTING_ID);
                return saved;
            });

        MedicalRecordResponse response =
medicalRecordService.create(validRequest);

        assertNotNull(response);
        assertEquals(EXISTING_ID, response.getId());
        assertEquals(PATIENT_ID, response.getPatientId());
        assertEquals("Caries dental", response.getDiagnosis());
        assertEquals("Resina compuesta", response.getTreatment());
        assertEquals("Control en 6 meses", response.getObservations());
        verify(medicalRecordRepository,
times(1)).save(any(MedicalRecord.class));
    }
```

Caso borde: entrada válida en el límite inferior aceptable

2. Verifica que buscar una historia clínica que no existe lance NotFoundException en vez de retornar null

Código:

```
@Test
    @DisplayName("Lanza excepción al buscar una historia clínica inexistente")
    void cuandoHistoriaClinicaNoExiste_debeLanzarExcepcion() {
        when(medicalRecordRepository.findById(MISSING_ID))
            .thenReturn(Optional.empty());
```

```

        NotFoundException ex = assertThrows(
            NotFoundException.class,
            () -> medicalRecordService.findById(MISSING_ID));

        assertEquals("Historia clínica no encontrada con id: " +
MISSING_ID,
            ex.getMessage());
    }

```

Caso borde: Id inexistente

3. Verifica que eliminar una historia inexistente lance la excepción y nunca invoque deleteById.

Código:

```

@Test
@DisplayName("No elimina y lanza excepción cuando la historia
clínica no existe")
void
cuandoSeEliminaHistoriaInexistente_debeLanzarExcepcionYNoEliminar() {
    when(medicalRecordRepository.existsById(MISSING_ID))
        .thenReturn(false);

    NotFoundException ex = assertThrows(
        NotFoundException.class,
        () -> medicalRecordService.delete(MISSING_ID));

    assertEquals("Historia clínica no encontrada con id: " +
MISSING_ID,
        ex.getMessage());
    verify(medicalRecordRepository,
never()) .deleteById(MISSING_ID);
}
}

```

Caso borde: intento de eliminar un id que no existe

FUNCIONAMIENTO DE LOS 3 TESTS + COVERAGE

